# Breaking the Memory Wall in GPU-Accelerated ANN Graph Construction

Anonymous Author(s)

## Abstract

Constructing graph-based indexes for approximate nearest neighbor search (ANNS) at scale demands massive high-dimensional distance evaluations that benefit substantially from GPU parallelism. However, when the working set exceeds device memory, construction must be partitioned into device-sized shards, incurring frequent data movement across the memory hierarchy and serialized host-side coordination that can together dominate end-to-end time.

We present SHEAR, a GPU-accelerated ANN graph construction system that organizes construction into memory-bounded subtree tasks, each formed by overlapping hierarchical partitioning to co-locate potential neighbors. Within each subtree, TESSA, a persistent tensor core engine, jointly executes recursive partitioning and pairwise distance evaluation under a unified 8-bit quantization scheme, emitting compact per-source candidate neighbors rather than materialized local subgraphs. On the host side, TierLift, an online neighbor selection algorithm, incrementally selects final graph edges from arriving candidates under a bounded memory budget, running concurrently with the GPU execution of subsequent subtrees and spilling excess state to SSD when DRAM is insufficient. Such a pipelined architecture replaces serialized cross-partition graph merging with incremental asynchronous CPU-GPU execution. On four public datasets with up to 100 million vectors using a single NVIDIA GeForce RTX 4090, SHEAR achieves 1.38–10.04× end-to-end construction speedups over three GPU-based baselines while preserving comparable accuracy-efficiency tradeoffs for ANNS.

***Keywords:*** Approximate Nearest Neighbor Search, Graph Index, GPU Acceleration, Vector Databases

## 1 Introduction

Approximate nearest neighbor search (ANNS) is a fundamental primitive in modern vector databases [42], with broad applications in retrieval-augmented generation (RAG), recommender systems, and image retrieval [5, 24, 25]. To deliver high-efficiency search, ANNS systems typically construct specialized vector indexes (*e.g.,* , trees, inverted files, and graphs) to prune the search space and bypass exhaustive scanning over the dataset [7, 20, 34, 36]. Among existing indexing paradigms, graph-based indexes have emerged as the de facto standard for ANNS due to their superior capability in navigating high-dimensional vector spaces, achieving an unparalleled trade-off between query throughput and search accuracy (recall) [11, 48, 57].

Building graph-based indexes is exceptionally resource-intensive in terms of both compute cycles and memory consumption. During construction, the system must perform exhaustive or iterative k-nearest-neighbor (kNN) searches and distance computation for every incoming node to establish multi-hop graph edges, resulting in high I/O memory bandwidth consumption and heavy CPU usage [52]. While graph-based indexes like HNSW [34], Vamana [48], and NN-Descent [8] vary in graph topology, their construction inherently couples compute-intensive distance evaluations with irregular graph-state operations (*e.g.,* , candidate pruning and adjacency updates). This inter-dependency enforces strict sequential ordering between execution steps, causing severe thread contention and parallel bottlenecking.

Fortunately, these two components match distinct hardware capabilities: distance evaluations possess massive data parallelism well-suited for GPUs [22], whereas irregular graph updates involve non-coalesced memory accesses and dynamic branch logic better handled by CPUs [61]. Consequently, partitioning tasks across processors, *i.e.,* assigning distance computation to the GPU and topology maintenance to the CPU, presents a compelling heterogeneous paradigm.

Nevertheless, achieving seamless execution overlap between the host and device is non-trivial, as it requires minimizing both communication latency and synchronization overheads. Existing heterogeneous index builders, such as CAGRA [41], couple the processors through iterative graph-state exchange: the GPU searches the transient graph to yield candidate pools, while the CPU prunes candidates and updates graph topologies, after which the updated graph is fed back to the GPU. This ping-pong feedback loop enforces lock-step synchronization, throttling overall search throughput down to the latency-heavy graph-update phase[1].

The problem further worsens as dataset sizes scale beyond GPU memory capacity. Under such out-of-core regimes, current systems adopt a divide-and-conquer strategy to construct the graph: they partition the dataset into device-sized chunks, build local subgraphs per partition on the GPU, and merge these partial results into a unified global graph [27, 39].

[1]It worth mention that, some recent work [13, 19, 27, 61] attempt to consolidate the entire workflow within GPU VRAM, so as to avoid CPU-GPU communication and synchronization overheads. They handle graph traversals via massive vector batching and manage topologies using fixed-size HBM arrays with Warp-level atomic primitives. However, forcing the GPU to execute irregular pointer-chasing and non-sequential edge pruning triggers severe control-flow divergence and uncoalesced memory accesses. This leads to heavy HBM bandwidth starvation and intra-device resource contention, leaving substantial GPU compute capacity underutilized.

To enable GPU–CPU pipeline overlap, state-of-the-art out-of-memory GPU builders perform this merging incrementally on the host [27, 30, 39]. Each GPU-built local subgraph is streamed to the host and merged into the global graph. However, merging subgraphs with overlapping vertices requires synchronized updates to shared adjacency lists, forcing execution onto a serialized path that fails to scale across multi-core CPUs. Consequently, the slower host-side processing rapidly saturates input buffers, triggering backpressure that halts GPU compute streams and destroys inter-processor pipelining. As an example, running Tagore-Vamana [27] on DEEP100M shows that host-side merge accounts for 41.3% of total construction time, *i.e.,* a light compute workload on the CPU consumes nearly half of the total latency.

This analysis reveals two coupled inefficiencies in existing out-of-memory GPU construction architectures. First, GPU-side partition builders interleave dense distance evaluations with irregular neighbor-list maintenance [27, 41], leaving the device's regular compute capacity underutilized. Second, host-side consolidation of complete subgraphs enforces a serialized merge path that fundamentally inhibits multi-core scalability. Both bottlenecks stem from the same root cause: existing paradigms insist on building complete local subgraphs within each GPU partition, which couples regular computation with irregular graph management on-device and yields coarse intermediate outputs that bottleneck host-side integration. Addressing this mismatch demands a reimagined GPU–CPU collaboration. By restricting the GPU strictly to regular distance computations, systems can fully exploit matrix-arithmetic throughput; by exchanging fine-grained candidate streams instead of coarse subgraphs, the host can perform independent, dependency-free reductions that completely eliminate host-side merge bottlenecks.

Based on this observation, we present SHEAR, a single-machine GPU–CPU graph construction system for datasets exceeding GPU memory capacity. SHEAR features a natively pipelined GPU–CPU framework in which the two processors are decoupled by design rather than coordinated via iterative state exchange. SHEAR constructs the graph through hierarchical partitioning, in-leaf candidate generation, and source-local edge reduction. Specifically, the dataset is recursively partitioned into overlapping leaf clusters to co-locate potential neighbors; pairwise distances within each leaf then yield per-source candidate edges, which are subsequently reduced into final adjacency lists. This workflow naturally matches the heterogeneous hardware – the partitioning and pairwise distance stages consist solely of regular matrix operations that saturate GPU Tensor Cores. In contrast, the subsequent edge reduction is source-local without shared mutable state, scaling linearly across CPU threads. Because the GPU's computation depends strictly on partition structures rather than the host's evolving graph state, the two processors operate asynchronously over a unidirectional candidate stream, achieving continuous pipeline overlap.

To realize this workflow efficiently at scale, SHEAR introduces three key design decisions. First, since individual subtrees must fit within GPU HBM, SHEAR forms memory-bounded subtree tasks through overlapping $k$-means assignment and encodes all vectors into a compact scalar quantization (SQ8) representation at the outset. This single encoding eliminates redundant data transformations and enables all subsequent GPU distance computation to execute through high-throughput tensor arithmetic. Second, SHEAR introduces TESSA, a persistent GPU engine that fuses hierarchical partitioning and leaf-level pairwise distance into a single long-running kernel. Rather than launching separate kernels for partitioning and candidate generation, TESSA keeps both workloads concurrently resident, allowing leaf-level candidate computation to absorb GPU cycles as soon as partitioning produces leaf clusters, without kernel-launch overhead or global synchronization. Third, on the host side, SHEAR introduces TierLift, an online reducer that performs direction-aware edge selection independently for each source under a bounded degree budget. Because each source is reduced in isolation, the host achieves lock-free thread-level parallelism that scales with core count, and when candidate states overflow DRAM, they are sequentially spilled and replayed without random I/O. Together, these designs enable SHEAR to overlap GPU subtree execution with host-side reduction continuously, sustaining high throughput on both processors simultaneously.

On four public benchmarks of 35M–100M vectors with a single RTX 4090, SHEAR achieves 1.38–10.04× end-to-end construction speedups over GPU baselines while preserving comparable Recall-QPS trade-offs.

- We present SHEAR, a single-machine GPU–CPU graph construction system for beyond-GPU-memory datasets. SHEAR replaces coarse-grained graph-state exchange with source-keyed candidate streaming, decoupling GPU candidate generation from host-side graph reduction.
- We design TESSA, a persistent Tensor Core engine that uses a unified SQ8 distance path for recursive partitioning decisions and in-leaf candidate generation.
- We propose TierLift, a memory-bounded candidate pruning mechanism, and design hybrid DRAM/SSD reduction paths for source-aware candidate state management under limited host-memory budgets.
- We evaluate SHEAR on four large-scale benchmarks. It achieves 1.73–10.04× end-to-end construction speedups over graph-based baselines, with complete Recall-QPS trade-offs reported for the resulting indexes.

## 2 Background

### 2.1 Basics of ANNS and Graph Index

**Approximate Nearest Neighbor Search:** Let $X = \{x_i\}_{i=1}^{N} \subset \mathbb{R}^d$ denote $N$ data objects in $d$-dimensional space, $q \in \mathbb{R}^d$

be a query vector, and $K$ be the number of neighbors to retrieve. This paper measures the distance between objects using squared Euclidean distance $\mathrm{dist}(x_i, x_j) = \|x_i - x_j\|_2^2$, which induces the same neighbor ordering as Euclidean distance. The resulting exact $K$-nearest-neighbor set is $R_K(q) = \mathrm{TopK}_{x \in X}\{-\mathrm{dist}(q, x)\}$. Computing $R_K(q)$ exactly requires comparing $q$ against all $N$ objects, incurring a complexity of $O(Nd)$. Approximate nearest neighbor search (ANNS) uses an index to reduce the search space, trading exactness for reduced search cost, and returns an approximate size-$K$ result set $R'_K(q) = \{x'_1, \ldots, x'_K\} \subseteq X$. The quality of an ANNS result is measured by $\mathrm{Recall@}K = |R_K(q) \cap R'_K(q)|/K$, i.e., the fraction of the objects in the approximate result that are also in the exact result.

**Graph Index for ANNS:** A graph index for ANNS (a.k.a. ANN graph) represents the dataset $X$ as a directed proximity graph $G = (X, E)$, where the adjacency list of any vertex $u$ is denoted $N_G(u)$ and satisfies the out-degree bound $|N_G(u)| \le R$. In traversal-based graph construction [34, 48], the builder searches the current graph to find the candidate set $C(u)$ for vertex $u$, then prunes and updates its adjacency list as $N_G(u) = \mathrm{Prune}\big(u, C(u), R\big)$. Since the search depends on the current graph and the adjacency updates, which in turn alter the graph on which subsequent searches rely, this process forms a data-dependent feedback loop. Such coupling produces irregular graph traversal, control flow, and memory access, which are difficult to organize directly into regular GPU batches [61]. To reduce the cost of repeated distance evaluations, compressed representations such as scalar quantization are often used to approximate distances during construction and search [1].

## 2.2 GPU Architecture

**Processing Units and Thread Hierarchy:** The GPU is a processor architecture designed for massively parallel computation. In a discrete-GPU setting, the CPU serves as the host and exchanges data with the GPU device over an interconnect such as PCIe. The GPU comprises multiple streaming multiprocessors (SMs), each equipped with CUDA cores, tensor cores, registers, and on-chip resources such as shared memory. Execution follows the single instruction, multiple threads (SIMT) model in which groups of threads execute the same instruction. Threads launched by a kernel are organized into cooperative thread arrays (CTAs), and each CTA is further partitioned into warps, typically consisting of 32 threads. Once a CTA is dispatched to an SM, the SM schedules instructions at warp granularity, switching among ready warps to hide computation and memory-access latency. As threads within a warp execute the same instruction, divergence into different control-flow branches forces the divergent paths to be executed separately, reducing effective parallelism.

**Tensor Core Matrix Multiplication Accelerators:** Tensor cores are dedicated computation units within the SM for matrix multiply-accumulate (MMA) operations subject to fixed tile shapes and data layout requirements. CUDA programs can access this functionality through several layers: the warp-level WMMA API, lower-level MMA instructions exposed directly by the hardware, and the CUTLASS [38] template library built atop these primitives, supporting mixed-precision inputs such as FP16, BF16, and TF32 with FP32 accumulation, as well as signed-INT8 inputs with INT32 accumulation.

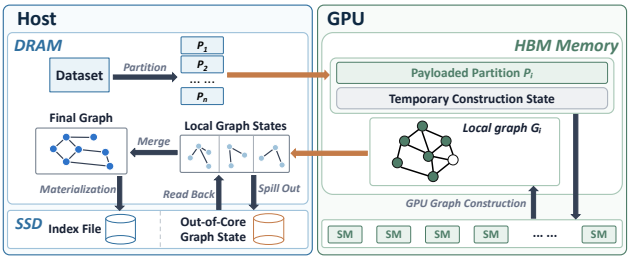



**Figure 1.** Execution flow of host-GPU graph construction.

## 2.3 GPU Memory-Bounded Graph Construction

As ANN graphs scale to datasets that exceed GPU HBM capacity, constructing graph indexes entirely within GPU HBM becomes challenging. Existing systems thus adopt memory-bounded GPU construction schemes, where the dataset is divided into HBM-sized partitions and processed sequentially or asynchronously. Figure 1 illustrates a typical host-GPU collaborative pipeline, where the host manages data partitions and intermediate graph states in DRAM, schedules GPU execution, and coordinates the merge of partition-level local graphs generated by the GPU. Because each local graph only captures intra-partition relationships, combining them into a global graph requires additional processing, such as adjacency list merging, duplicate removal, and degree-constrained pruning. When intermediate graph states exceed DRAM capacity, they will be spilled to SSD and later read back during global graph recovery.

Heterogeneous pipelines improve construction efficiency by overlapping GPU local graph generation with host-side integration of partition results, thereby reducing part of the host-side processing overhead. However, the effectiveness of such overlap depends on the throughput balance between GPU result production and host-side consumption. Our profiling of Tagore-Vamana [27] on the DEEP100M dataset [3] reveals that local construction accounts for only about 30.7% of the total GPU time, while GPU stalls due to producer-consumer imbalance and post-GPU processing contribute 13.8% and 41.3%, respectively. The primary cause is that Tagore-Vamana organizes local graph states by partitions, whereas concurrent merging is limited because different partitions may contain updates for the same source vertex, yielding conflicts on adjacency lists and requiring serialized processing. Furthermore, when GPU production exceeds

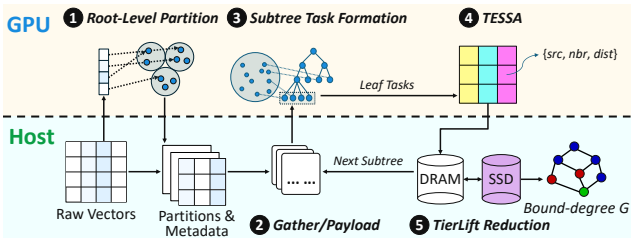



**Figure 2.** Pipeline of the SHEAR framework.

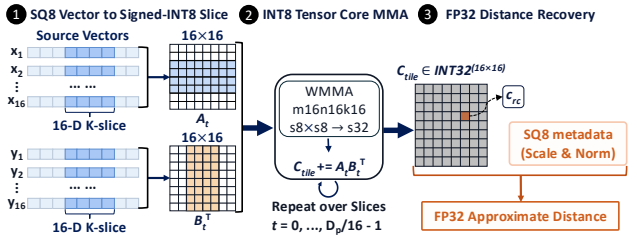



**Figure 3.** Illustration of the data path for approximate distance evaluation using TESSA.

CPU merge throughput, host-side buffers become saturated because each slot is released only after the corresponding graph state is merged or spilled to SSD, forcing subsequent GPU tasks to wait for available slots. Finally, online merging does not directly produce the final index representation. After GPU execution completes, the system must scan partition-level intermediate states, compact valid adjacency lists, and materialize the global index, resulting in an unavoidable post-GPU tail.

# 3 SHEAR Design

## 3.1 Framework Overview

To address the limitations of existing GPU-accelerated methods for ANN graph construction, we propose SHEAR in this section. As illustrated in Figure 2, the SHEAR pipeline consists of three stages: (1) **preparation** that concurrently derives GPU and CPU metadata to form admissible subtree tasks under GPU HBM constraints; (2) **subtree construction** that jointly advances recursive partitioning and in-leaf candidate generation on the GPU, reusing the TESSA core for distance evaluation and emitting compact candidate records instead of materializing local graphs; (3) **host-side reduction** that continuously reduces candidate states under host DRAM constraints while managing overflow states through SSD-backed storage for global graph materialization. Moreover, SHEAR employs buffering that allows host-side processing and data movement to overlap with GPU processing of subsequent subtrees, thereby improving resource utilization.

## 3.2 Subtree Task Formation

To decouple GPU execution granularity from dataset scale, SHEAR first forms GPU-admissible subtree tasks through root-level partitioning, where $K_0$ centers $\mathcal{M}_0 = \{\mu_1, \ldots, \mu_{K_0}\}$ are computed by k-means clustering on sampled data to capture the global vector distribution. Then, each point $x_i$ is assigned to its $r_0$ nearest centers, i.e.,

$$A_0(x_i) = \mathrm{Top}_{r_0, \mu \in \mathcal{M}_0}\big(-\widetilde{D}(x_i, \mu)\big),$$

where $\widetilde{D}(\cdot,\cdot)$ denotes the SQ8 approximate distance evaluated by the TESSA core (as detailed in §3.3). The subtree associated with center $\mu_j$ consists of all points with $\mu_j \in A_0(x_i)$.

The partitioning permits overlap, allowing a point to belong to multiple subtrees. This can improve the probability that potential neighbors remain covered by at least one subtree. In the partitioning, the parameter $K_0$ determines the subtree granularity, and the parameter $r_0$ balances neighborhood coverage and replication overhead. Subtrees that exceed the target size or fail admission are lazily split on demand before being admitted to the backend.

The partitioning process handles the dataset in batches concurrently. The GPU generates 8-bit SQ codes as a compact representation, which are consumed by TESSA for root-level assignment immediately after encoding and reused for subsequent subtree-level distance evaluation. Meanwhile, the CPU concurrently computes low-dimensional projection sketches, which are later used by TierLift to derive residual-direction hashes and incrementally reduce candidate streams into bounded per-source adjacency lists. This process produces two types of persistent metadata: (1) per-center subtree membership records and (2) a globally indexed SQ sidecar containing quantized codes, scales, and norms, which respectively serve as input for subsequent execution.

## 3.3 Subtree Construction

Each admitted subtree task is asynchronously staged to the GPU through a bounded host-side preload queue. The queue gathers the quantized subtree payload from the SQ sidecar by member ID and overlaps the preparation of the next subtree with the GPU execution of the current one. The loaded SQ codes are then consumed by the persistent kernel. The kernel jointly executes recursive partitioning and in-leaf candidate generation while sharing the TESSA core for approximate distance evaluation.

Within each subtree, load-imbalanced recursive partitioning incrementally produces leaf tasks that are consumed by dense in-leaf top-$k$ candidate computation. A conventional two-stage schedule executes partitioning and candidate generation in separate kernel phases, preventing leaf tasks from being consumed until partitioning is complete. SHEAR's fused-persistent execution instead keeps partition and candidate CTAs concurrently resident within the same persistent kernel, allowing the two workloads to overlap under dynamically varying partition loads. Within such an execution model, the TESSA core forms SQ8 vector-pair tiles, performs signed-INT8 WMMA with tensor core INT32 accumulation, and reconstructs FP32 distances for both workloads.

**TESSA for SQ8 Distance Evaluation:** As both partition CTAs and candidate CTAs in the persistent kernel repeatedly evaluate large numbers of vector-pair distances, SHEAR introduces the TESSA core as a unified Tensor Core-based engine for efficient distance computation. Figure 3 illustrates the approximate distance evaluation using the TESSA core. Given SQ8-encoded vector pairs, a TESSA core performs accelerated dot product computation and reconstructs FP32 approximate distances using accumulated products, quantization scales, and original norms. The TESSA core is the shared SQ8 distance evaluation engine used by recursive assignment and in-leaf top-$k$ candidate computation.

For a recursive node at depth $h$, an assignment evaluates an $m_h \times \ell_h$ distance block between its $m_h$ members and $\ell_h$ sampled leaders, where $m_h$ decreases with recursive refinement and $\ell_h$ determines the local branching factor. For a leaf cluster containing $m_l$ points, candidate computation evaluates the corresponding pairwise distance space of size $m_l \times m_l$ to derive per-source top-$k$ candidates. Although the two workloads differ in input objects and output shapes, they both compute squared $\ell_2$ distances following $D_{ij} = n_i + n_j - 2x_i^\top x_j$, where $n_i = \|x_i\|_2^2$. With norms precomputed, their dominant arithmetic operations reduce to blocked dot products, allowing both workloads to reuse the same SQ8 distance computation path.

The SQ8 representation used by TESSA is generated once before GPU execution. Each vector is quantized using a symmetric per-vector scale $s_i$, where $s_i = a_i/127$ with $a_i = \max_t |x_{it}|$ when $a_i > 0$, and $s_i = 1$ otherwise. Each coordinate is then quantized as

$$\hat{x}_{it} = \mathrm{clip}(\mathrm{round}(x_{it}/s_i), -127, 127) .$$

The scaling factor $s_i$ and original norm $n_i$ are stored as FP32 metadata, and the approximate distance is recovered as

$$\widetilde{D}_{ij} = \max\big(0,\ n_i + n_j - 2s_i s_j(\hat{x}_i^\top \hat{x}_j)\big) .$$

SHEAR generates this encoding once at the root level and reuses it as input to the TESSA core for root-level assignment, recursive assignment within partitioning, and in-leaf top-$k$ candidate computation. The SQ8 representation reduces the vector storage footprint to approximately one quarter of the FP32 representation, reducing data movement during sidecar aggregation, host-to-GPU data transfer, and memory access.

For tensor core execution, TESSA groups source and target vectors into 16×16 INT8 operand tiles. Each tile pair produces a $16 \times 16$ dot-product tile containing 256 source-candidate dot products, which are accumulated across consecutive 16-dimensional slices. The SQ8 path uses `m16n16k16` signed-INT8 MMA with INT32 accumulation; the tensor core baseline path, which takes the original FP32 vectors as input, uses `m16n16k8` TF32 MMA with FP32 accumulation. The former covers twice the $K$ dimension per MMA operation, reducing the number of accumulation steps required for the same dot-product tile.

After the dot-product accumulation, TESSA reconstructs FP32 approximate distances online. SHEAR then applies task-specific epilogues: the partition epilogue maintains each member's nearest top-$r_h$ leader assignments, whereas the candidate epilogue filters self-pairs and updates the per-source-point top-$k$ state. Both epilogues consume recovered distances tile by tile, avoiding materialization of the full logical distance matrix in device memory.

**Partition-Candidate Co-Execution:** To avoid waiting for complete subtree partitioning before candidate generation, SHEAR executes recursive partitioning and in-leaf candidate computation as a producer-consumer pipeline within a persistent kernel. The kernel maintains two device-resident queues: a partition queue $Q_p$ holding nodes awaiting partitioning and a leaf queue $Q_l$ holding published leaf tasks; the root node is enqueued into $Q_p$ at launch. A subset of resident CTAs acts as partition workers that dequeue nodes from $Q_p$ and advance the recursion, returning non-terminal children to $Q_p$ for further partitioning and publishing terminal children to $Q_l$ as leaf tasks. The remaining resident CTAs act as candidate CTAs that dequeue available tasks from $Q_l$ to perform in-leaf top-$k$ candidate computation. Because leaf tasks can be consumed immediately after publication, recursive partitioning and in-leaf top-$k$ candidate computation overlap on the device without waiting for subtree-wide partition completion or requiring host-side synchronization. The kernel terminates only at global quiescence, when both $Q_p$ and $Q_l$ are empty and no active work remains.

Unlike the one-time root-level k-means partitioning, partition workers recursively apply Randomized Ball Carving (RBC) to refine admitted subtrees into leaf-sized clusters, as repeatedly applying k-means would introduce substantial clustering overhead during recursive refinement, making RBC a more lightweight alternative [45]. At depth $h$, a node samples $\ell_h$ leaders and retains each member's nearest $r_h$ leader assignments; only the first split permits multiple assignments to improve boundary coverage, while deeper levels set $r_h = 1$ to bound recursive replication.

Recursive partitioning may generate highly imbalanced nodes, where a small number of large nodes dominate partitioning cost. To mitigate this load imbalance, SHEAR divides the members of large nodes into 16-row work blocks and processes them cooperatively across multiple CTAs using COUNT-SCATTER. During COUNT, the partition workers invoke the TESSA core to evaluate member-leader distances. SHEAR's partition epilogue then selects and caches each member's $r_h$ assignment labels while accumulating the size of each child node.

Each 256-thread partition CTA comprises eight warps, with one work block assigned to each warp. With batch-1, the CTA claims one block at a time; with batch-8, it claims up to eight consecutive blocks, improving warp utilization. After COUNT completes, SHEAR derives each child node's

output interval from the accumulated counts. SCATTER then reuses the cached assignment labels to write members into their corresponding child-node sequences; caching the labels uses $O(m_h r_h)$ temporary storage and avoids distance recomputation. SCATTER uses the same batched-claim mechanism to reduce claims and amortize work-claim overhead. When $Q_l$ has no ready leaf work, candidate CTAs may assist large-node partitioning to mitigate global CTA-level imbalance.

**Compact Candidate Output:** To reduce GPU memory pressure and irregular per-source graph state maintenance, SHEAR performs only local top-$k$ selection on the device and emits compact candidate records for host-side reduction. Once a node reaches the leaf-level size, its members serve as both source and candidate points. TESSA evaluates their pairwise distances tile by tile, and the candidate epilogue incrementally maintains a local top-$k$ list for each source point. A single CTA may own the top-$k$ buffers for a group of source points and traverse the full candidate range; when the candidate range is partitioned across CTAs, each CTA first maintains a local top-$k$ buffer and then merges it to produce the final top-$k$ candidates for each source point.

After leaf processing is complete, candidate CTAs append the resulting top-$k$ entries as compact candidate records to the current subtree's device output array, where each record contains the source point's local row, the candidate's global ID, and the distance. Because the same source point may appear in multiple leaves or overlapping subtrees, these records may contain duplicates and intentionally do not form final adjacency lists; once the subtree is complete, the candidate batch is handed to the host for reduction. By performing local top-$k$ selection before output, SHEAR limits the generated data volume to $O(M_l k)$, where $M_l$ is the total number of leaf-member rows after recursive replication, instead of materializing the $O(M_l^2)$ in-leaf distance matrix.

### 3.4 TierLift for Candidate Reduction

As compact candidate batches arrive from completed GPU subtrees, SHEAR transfers them to the host and partitions them by global source ID, allowing host-side reduction to overlap with subsequent subtree execution. For each source point, the CPU incrementally reduces candidate records into a bounded neighbor set with a degree limit $R$. TierLift performs this reduction by deduplicating repeated targets and maintaining a primary candidate set $P$ of directionally diverse neighbors using residual-direction hash buckets. A strict one-representative-per-bucket policy, however, may discard useful same-bucket alternatives and leave part of the available degree budget unused. TierLift, therefore, maintains bounded collision alternatives from occupied buckets and selectively admits high-quality candidates according to the remaining degree budget and recovery cap $\tau$. The detailed procedure is shown in Algorithm 1.

**Algorithm 1** TierLift Online Candidate Reduction

**Input:** A stream of candidates $\mathcal{S}$; degree bound $R$; recovery cap $\tau$; tier penalty $\alpha$
**Output:** Ordered adjacency sequence $\Gamma$

$P, C_{\text{col}} \leftarrow \emptyset$
**for all** $e \in \mathcal{S}$ **do**
  **if** the hash bucket of $e$ is new **then**
    /*Add $e$ as a new primary representative*/
    $P \leftarrow P \cup \{e\}$
  **else if** $e$ improves the current primary candidate **then**
    /*Replace primary $p$ and retain it as collision candidate*/
    $P \leftarrow P \setminus \{p\} \cup \{e\}$ and $C_{\text{col}} \leftarrow C_{\text{col}} \cup \{p\}$
  **else**
    /*Store $e$ as a collision candidate*/
    $C_{\text{col}} \leftarrow C_{\text{col}} \cup \{e\}$
$B \leftarrow \min\{\tau, R - |P|\}$
/*Select recovery candidates under degree bound*/
$r \leftarrow \text{Top}_B(C_{\text{col}}; \delta\alpha^{2(j-1)}, j, v)$
**return** $\Gamma = P \parallel r$

**Bounded Collision-Candidate Recovery:** Each candidate record initially contains a source identifier, a neighbor ID, and its distance. Upon receiving a candidate $e = (u, v, \delta)$, the host routes it to the TierLift candidate reservoir associated with source $u$.

TierLift derives the residual-direction hash

$$\phi_u(v) = \text{sign}(\text{Sketch}(v) - \text{Sketch}(u))$$

from precomputed directional sketches and uses the resulting bit pattern as the candidate's bucket identifier. The hash captures the residual direction from $u$ to $v$, grouping candidates with similar directions into the same bucket.

Within each source-specific reservoir, TierLift maintains a primary set $P$ with at most one representative per retained hash bucket. If the bucket is unseen, the candidate is inserted as its primary representative. For an occupied bucket, the closer candidate becomes the primary representative, while the displaced or losing candidate is retained in the bounded collision set $C_{\text{col}}$.

The collision set $C_{\text{col}}$ stores discarded candidates that are potentially useful for graph connectivity. TierLift removes duplicate targets and retains a bounded number of nearest candidates for each hash bucket. It then ranks candidates in $C_{\text{col}}$ using the tier-aware key $(\delta\alpha^{2(j-1)}, j, v)$ and selects at most $\min\{\tau, R - |P|\}$ candidates as the recovery set $r$. The factor $\alpha$ favors shallower tiers while allowing sufficiently closer candidates from deeper tiers to take precedence.

The final adjacency sequence is constructed as $\Gamma = P \parallel r$, which satisfies $|\Gamma| - |P| \le \tau$ and $|\Gamma| \le R$. In this way, TierLift selectively exploits the remaining degree capacity rather than filling the adjacency list to the degree bound $R$.

**Table 1.** Statistics of datasets used in the experimental evaluation. Size is the decimal size of the float32 base-vector file.

| Dataset | Modality | Dim. | #Vectors | #Queries | Size (GB) |
|---|---|---|---|---|---|
| DEEP100M [3] | Image | 96 | 100M | 10K | 38.40 |
| LAION100M [46] | Image | 512 | 100M | 10K | 204.80 |
| MSMARCO70M [4] | Text | 768 | 70M | 10K | 215.04 |
| Wiki35M [6] | Text | 1024 | 35M | 10K | 143.36 |

### 3.5 Out-of-Core Reduction

For large-scale datasets, the host DRAM cannot always accommodate candidate reservoirs for all source points. Thus, SHEAR manages candidate reservoirs using a tiered residency strategy, where a portion of source ranges remains resident in memory, and overflow candidate records are sequentially appended to SSD files partitioned by source-ID ranges for later reduction.

Under memory constraints, the host budget is shared between the SQ sidecar and candidate reservoirs. SHEAR prioritizes SQ-sidecar residency because encoded vectors are repeatedly gathered by admitted subtree tasks, and nonresident accesses would introduce random external-memory latency into the GPU execution path. In contrast, candidate reservoirs are produced as a sequential stream and do not require immediate reduction. When the reservoir capacity is insufficient, excess candidate records are sequentially spilled by source-ID order and replayed through external reduction, avoiding random disk access during out-of-core processing.

After candidate generation, resident and out-of-core candidate reservoirs are reduced using the same source-aware reduction procedure. Candidates are grouped by source ownership, where records belonging to the same source are reduced exclusively while disjoint source ranges can be processed concurrently. For resident states, CPU workers consume GPU-generated candidate batches through a byte-bounded queue and update in-memory reservoirs according to source ownership. The worker degree is adjusted according to producer-consumer throughput, and queue backpressure bounds in-flight candidate memory while overlapping GPU candidate generation with CPU reduction. For out-of-core states, the reducer replays source-partitioned spill records and reconstructs bounded reservoirs within temporary memory. By assigning disjoint source-ID ranges to workers during replay, SHEAR enables parallel out-of-core reduction without fine-grained synchronization.

## 4 Evaluation

The experimental evaluation aims to answer the following four research questions:

- **RQ1:** Does SHEAR reduce the time taken to construct an ANN graph from raw vectors compared to existing GPU-accelerated construction approaches?
- **RQ2:** Can SHEAR preserve comparable index quality in terms of the accuracy-efficiency tradeoff for ANNS?
- **RQ3:** How much do GPU memory-aware task formation, TESSA, subtree-level persistent execution, compact candidates, and TierLift contribute to overall performance?
- **RQ4:** Under a tight host memory budget, how can reuse-aware residency and out-of-core reduction lower DRAM and I/O overhead while preserving index quality?

### 4.1 Experimental Setup

**Environment:** All experiments were run on a server with a 32-core AMD(R) Ryzen™ Threadripper™ PRO 7975WX CPU, 256 GiB of DRAM, and an NVIDIA(R) GeForce RTX™ 4090 GPU with 24 GiB of device memory. The server ran Linux 6.8 and CUDA 13.0 and used local NVMe storage.

**Datasets:** We performed experiments on four benchmark datasets. Table 1 summarizes the dataset statistics. The ablation studies used LAION35M and Wiki10M, which are randomly sampled subsets of LAION100M and Wiki35M, respectively. All vectors retain their original float32 precision, and the Euclidean distance is used as the distance metric.

**Baselines:** We compare SHEAR with the following four baselines. CAGRA builds a local graph in each ACE shard with the cuVS NN-Descent solver; Tagore-CAGRA instead uses Tagore for in-shard construction; and Tagore-Vamana constructs a Vamana graph per shard and merges the shard graphs into a global index on the CPU [27, 41]. IVF-RaBitQ-GPU constructs a quantized IVF index on the GPU [12]. Note that all searches on each index are performed on a single CPU thread.

### 4.2 Experimental Results

**Overall Performance (RQ1&2):** To analyze end-to-end overhead, we divide the construction process into three stages, as in Table 2. SHEAR achieves the fastest end-to-end construction among all graph construction methods across four datasets, providing 1.38–2.55× speedups over the fastest baselines and 3.79–10.04× speedups over Tagore-Vamana.

The stage breakdown reveals that SHEAR's advantage mainly comes from reduced core construction and post-processing overhead. For CAGRA and Tagore-CAGRA, the post-processing time is negligible because they adopt a non-overlapping partitioning strategy that avoids merging duplicate local graph states across partitions. Tagore-Vamana suffers from both expensive core construction and post-processing overhead. Its complex pruning and graph refinement increase the core construction cost, while its CPU-side merging remains on the critical path after GPU execution. Quantization-based IVF-RaBitQ is constructed faster than graph-based SHEAR on the GPU.

Figure 4 presents the tradeoff between Recall@10 and QPS for each method across the four datasets. When Recall@10=0.95, SHEAR maintains QPSs comparable to or better than those of other graph indexes. The ANNS efficiency on IVF-RaBitQ is inferior to that of graph indexes on

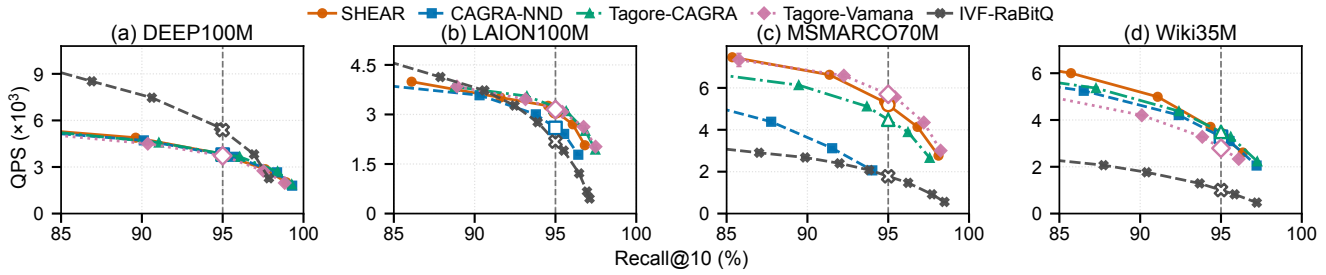



**Figure 4.** Recall@10 vs. QPS on the four datasets. The markers on the dashed lines denote the QPS of each method at a Recall@10 of 0.95. Note that CAGRA does not reach a 0.95 Recall@10 on MSMARCO70M after exhaustive traversal.

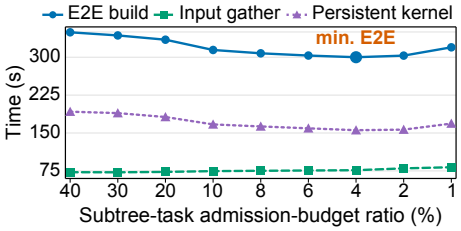



**Figure 5.** Effect of admission-budget ratio for subtree tasks on LAION35M.

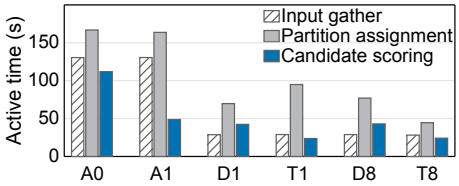



**Figure 6.** Stage-time breakdown of each ablation variant on Wiki10M.

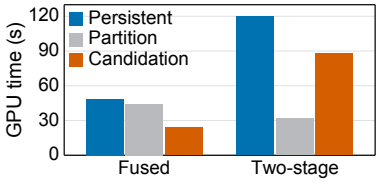



**Figure 7.** GPU time of fused-persistent vs. two-stage execution on Wiki10M.

**Table 2.** Overall construction and search performance of each method. The times taken for end-to-end construction (E2E), pre-processing (Pre), core construction (Core), and post-processing (Post) are in minutes.

| Dataset | Method | E2E | Pre | Core | Post |
|---|---|---|---|---|---|
| DEEP100M | SHEAR | **6.17** | 0.92 | 4.85 | 0.41 |
| | CAGRA | 17.99 | 0.84 | 17.15 | <0.01 |
| | Tagore-CAGRA | 8.53 | 0.85 | 7.69 | <0.01 |
| | Tagore-Vamana | 32.48 | 2.58 | 16.79 | 13.11 |
| | IVF-RaBitQ | 3.62 | — | 3.05 | 0.57 |
| LAION100M | SHEAR | **11.68** | 2.98 | 8.16 | 0.53 |
| | CAGRA | 27.63 | 6.08 | 21.55 | <0.01 |
| | Tagore-CAGRA | 25.31 | 6.25 | 19.05 | <0.01 |
| | Tagore-Vamana | 63.15 | 15.60 | 36.41 | 11.14 |
| | IVF-RaBitQ | 7.83 | — | 5.72 | 2.12 |
| MSMARCO70M | SHEAR | **11.62** | 1.76 | 9.49 | 0.37 |
| | CAGRA | 23.43 | 6.92 | 16.50 | <0.01 |
| | Tagore-CAGRA | 20.09 | 7.51 | 12.58 | <0.01 |
| | Tagore-Vamana | 43.99 | 13.13 | 25.57 | 5.29 |
| | IVF-RaBitQ | 8.43 | — | 6.18 | 2.25 |
| Wiki35M | SHEAR | **5.36** | 1.13 | 4.04 | 0.19 |
| | CAGRA | 13.68 | 3.80 | 9.88 | <0.01 |
| | Tagore-CAGRA | 15.37 | 3.77 | 11.60 | <0.01 |
| | Tagore-Vamana | 53.81 | 8.09 | 40.80 | 4.92 |
| | IVF-RaBitQ | 3.76 | — | 2.26 | 1.50 |

three high-dimensional datasets. These results confirm that SHEAR substantially reduces graph construction costs while preserving high graph quality for ANNS.

**Admission-Budget Ratio (RQ3.1):** To identify an appropriate admission-budget ratio, we vary it from 1% to 40% on LAION35M. As shown in Figure 5, reducing the ratio from 40% to 4% increases the speedups for end-to-end construction and persistent kernel execution, with the best performance achieved at 4%. This improvement comes from finer-grained subtree admission, which reduces the serialized recursive partition performed by each CTA. Larger admitted subtrees expose more internal nodes and require more row-leader assignment passes before producing sufficient leaf tasks, delaying top-$k$ CTA execution. Reducing the admission ratio shortens the serialized partition critical path, allowing leaf tasks to be generated earlier. However, reducing the ratio too much increases task management overhead, degrading execution efficiency. These results show that the admission-budget ratio affects the tradeoff between subtree granularity and management overhead, and we use the best-performing configuration for each dataset in the remaining experiments.

**Low-level Operator Optimization (RQ3.2):** To isolate the contribution of each component, we compare six paired SHEAR variants on Wiki10M. As shown in Figure 6, A1 replaces the SIMT distance backend of A0 with TF32 WMMA while keeping raw-FP32 inputs and batch-1 assignment unchanged. A1 achieves a 2.30× speedup in candidate generation with negligible impact on partition active time, showing that regular leaf-level top-$k$ processing directly benefits from tensor core execution. D1 adopts SQ8 sidecars and packed DP4A under batch-1 execution. For subtree input preparation, D1 achieves a 4.54× speedup over A1, demonstrating that compact SQ8 representation substantially reduces input materialization overhead. T1 replaces DP4A with TESSA

**Table 3.** Recall@10 and maximum losses vs. FP32 using different distance paths and expansion budgets $L$ on Wiki10M.

| Mode | $L = 50$ | $L = 100$ | $L = 200$ | $L = 400$ | Max Loss |
|---|---|---|---|---|---|
| FP32 | 0.8902 | 0.9396 | 0.9682 | 0.9847 | — |
| TF32 | 0.8902 | 0.9402 | 0.9689 | 0.9849 | 0.0000 |
| SQ8-WMMA | 0.8901 | 0.9400 | 0.9699 | 0.9853 | 0.0001 |
| SQ8-DP4A | 0.8912 | 0.9410 | 0.9693 | 0.9845 | 0.0002 |

while preserving the SQ8 representation and batch-1 execution. Compared with D1, T1 achieves a 1.79× speedup in candidate generation but increases partition active time to 1.36× that of D1. This indicates that TESSA accelerates dense distance computation, while batch-1 assignment provides insufficient matrix dimensions to fully utilize WMMA for irregular workloads. T8 addresses this limitation by increasing the large assignment batch size from 1 to 8 while keeping TESSA unchanged. Compared with T1, T8 reduces partition active time and persistent-event time by factors of 2.13 and 2.38, respectively, with little change in candidate generation time. In contrast, D8 shows negligible improvement over D1, suggesting that the batching benefit mainly comes from improving WMMA utilization under TESSA's assignment mapping. Under batch-8 execution, T8 reduces persistent-event time by 1.73× relative to D8. In general, SQ8, TESSA, and batch-8 address bottlenecks in input preparation, distance computation, and WMMA utilization, respectively, for irregular assignment workloads.

**SQ8 Distance Approximation (RQ3.3):** To assess whether SQ8 distance approximation affects graph quality, we compare three FP32 and SQ8 distance paths on Wiki10M, including the original FP32-SIMT path, the SQ8-based DP4A path, and the TESSA distance path. As shown in Table 3, replacing the FP32 distance path with SQ8 results in a maximum Recall@10 loss of at most 0.0002. The production TESSA configuration remains within 0.0005 of the FP32 baseline across all search budgets, demonstrating negligible impact on search quality. To isolate distance approximation from task-generation differences, we replay construction tasks collected from a production build through FP32 and SQ8 distance paths. SQ8 achieves mean per-task selected-ID overlaps of 99.36% for assignment and 99.21% for leaf top-$k$ candidate generation. The limited decision changes result from small SQ8-induced distance perturbations and are absorbed by the redundancy of multi-level fanout, where each point has multiple opportunities to encounter true neighbors in downstream leaf clusters. Together, these results show that SQ8 accelerates distance computation while preserving graph navigability and high-recall search quality.

**Fused-Persistent Subtree Execution (RQ3.4):** As shown in Figure 7, fused-persistent subtree execution achieves a 1.626× end-to-end construction speedup over two-stage execution. In the GPU execution path, two-stage serializes recursive partitioning and in-leaf candidate generation, while fused-persistent overlaps them within a single execution, reducing GPU time by 2.46×. This overlap removes the stage barrier and reduces 81.6% of the in-leaf top-$k$ active time from the GPU critical path. The in-leaf top-$k$ active time differs by less than 4% between the two schemes, while fused-persistent additionally uses 3.86M candidate-CTA assists during partition execution. These results show that fused-persistent shortens the GPU critical path by exploiting idle candidate CTAs to absorb partition-side work without increasing the overall candidate-generation workload.

**TierLift Pruning (RQ3.5):** To evaluate the impact of candidate ranking policies in TierLift, we fix $R = 64$ and $\tau = 3$ on LAION100M and compare Plain, DistanceFirst, HardTier, and TierLift, where TierLift uses $\alpha = 1.02$. All policies use the same recovery configuration and fixed budget, differing only in candidate ranking. Plain disables collision recovery as the baseline, DistanceFirst prioritizes distance quality when selecting recovered candidates, HardTier strictly favors lower collision tiers, and TierLift softly balances tier preference and distance quality using a tunable penalty. In Figure 8(a), we can see that compared with Plain, all Q3 policies improve the Recall-QPS tradeoff, demonstrating that supplemental candidate recovery enhances graph quality under a bounded degree budget. Among the Q3 variants, TierLift consistently outperforms HardTier and achieves a better tradeoff than DistanceFirst over a broad medium-to-high Recall range. TierLift selects supplemental edges with an intermediate balance between tier preference and distance quality, avoiding the overly strict selection of HardTier and the distance-only tendency of DistanceFirst. Figure 8(b) further examines the high-Recall region. At Recall@10 = 0.94, TierLift improves QPS by 2.15% over DistanceFirst, while the gap gradually narrows at higher Recall levels. These results show that TierLift achieves a balanced Recall-QPS tradeoff by allowing distance quality and tier preference to jointly influence candidate selection under a limited candidate budget.

**SQ vs. Candidate Reservoir Residency (RQ4.1):** To quantify the marginal benefit of using resident memory for the SQ sidecar and the candidate reservoir, we vary their resident fractions for evaluation. Figure 9 shows the performance improvement obtained from each additional 1 GiB of resident memory. Full-state and Partial-state denote configurations with full and approximately half residency, respectively. Under both settings, the SQ sidecar provides approximately 20× higher marginal benefit per GiB than the candidate reservoir. This results from their different access patterns. File-backed SQ rows are repeatedly gathered by overlapping subtrees and remain on the GPU-consumption critical path, making non-resident SQ accesses extremely expensive. In contrast, candidate states are generated as a sequential stream and can be deferred for later reduction, which is less sensitive to non-resident execution. These results motivate prioritizing SQ-sidecar residency under host-memory constraints.

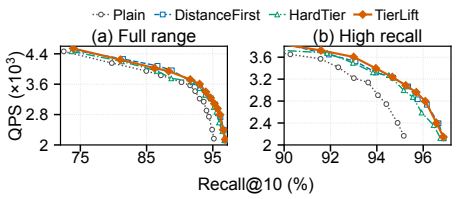



**Figure 8.** Recall-QPS tradeoff of TierLift ranking policies on LAION100M.

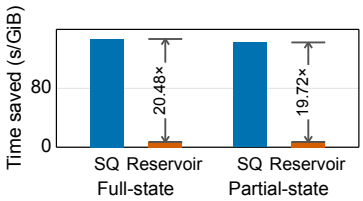



**Figure 9.** SQ vs. candidate reservoir residency on Wiki10M.

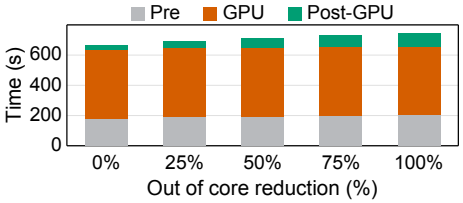



**Figure 10.** Stage-time breakdown of out-of-core reduction ratios on LAION100M.

**Out-of-Core Reduction (RQ4.2):** To evaluate the scalability of SHEAR under memory constraints, we vary the ratio of out-of-core candidate reduction states on LAION100M. Figure 10 presents the construction time breakdown as we vary from fully resident candidate reduction states to fully out-of-core ones. GPU denotes persistent-kernel execution time, while Post-GPU denotes the subsequent candidate-reduction phase on the host. Across all settings, the persistent-kernel execution time remains nearly constant, indicating that resident or out-of-core reduction does not affect GPU subtree construction. As the out-of-core ratio increases, the end-to-end construction time increases from 666 to 747 seconds, mainly due to additional post-processing costs. Even under fully out-of-core candidate reduction, SHEAR still preserves stable GPU construction and introduces limited host-side reduction overhead. These results demonstrate that SHEAR can scale beyond host memory capacity by spilling candidate states and performing out-of-core reduction.

## 5 Related Work

**ANN Graph Construction:** Graph indexes have been extensively used for ANNS in real-world vector-centric systems [42, 54]. Typical examples include NSW/HNSW [33, 34], Vamana [48], NN-Descent [8], NSG/NSSG [10, 11], and HVS [32]. The construction of all these ANN graphs involves repeated distance evaluations and neighbor selections, which are compute-intensive on large-scale data [2, 57]. Many methods accelerate ANN graph construction by refining graph structures [40, 51, 59, 60], partitioning the graph into small subgraphs [37, 45, 48], and exploiting CPU parallelism [9, 55]. Note that none of them considers utilizing GPUs for ANN graph construction.

**GPU-Accelerated ANN Graph Construction:** The massively parallel distance evaluations in ANN graph construction naturally benefit from GPU parallelism, motivating GPU-accelerated construction [13, 53, 61]. CAGRA [39, 41] builds an NSSG graph on the GPU and refines it into a fixed-outdegree structure optimized for GPU-based ANNS. Tagore [27] accelerates NN-Descent construction and extends it to out-of-core processing by streaming clusters via an asynchronous GPU-CPU-disk framework with cluster-aware caching. CMANNS [19] utilizes compute-memory disaggregation to optimize ANN graph construction on GPUs. Moreover, beyond GPUs, reconfigurable hardware, e.g., FPGAs [28, 29, 44], has also been utilized to accelerate ANN graph construction. Recently, a benchmark [30] observed that kernel-only speedups in these systems cannot be fully converted into end-to-end performance gains, as host-side partition management and graph integration constitute a substantial fraction of total construction time.

**Hardware-aware Graph-based ANNS:** There have been many studies on graph-based ANNS methods in various hardware configurations, rather than the vanilla setting of operating on a memory-resident graph using a single CPU thread. For example, ANNS methods on disk-resident graphs were studied in [14, 18, 26, 48, 56]. SSD-based ANNS methods were investigated in [16, 17, 47, 58]. Multi-core CPU approaches for ANNS were considered in [35, 43]. Finally, a large body of methods has been proposed to accelerate ANNS on the GPU [13, 15, 21, 22, 31, 50, 58, 61, 63–66], along with extensions to multi-GPU systems [23], CPU-GPU collaborations [49], and FPGAs [62]. These studies are orthogonal to this work because they focus on ANNS on pre-built graphs rather than graph construction.

## 6 Conclusion

Scaling graph construction for ANNS beyond GPU memory requires rethinking how GPU computation, graph state exchange, and host-side reduction are organized under constrained device and memory budgets. We present SHEAR, a GPU-CPU collaborative system that replaces coarse-grained local graph exchange with fine-grained candidate streaming for ANN graph construction. The memory-bounded subtree execution model keeps GPU workloads within a fixed HBM envelope and continuously generates compact candidate streams without materializing intermediate graphs. TESSA unifies recursive partitioning and leaf-level candidate generation through a persistent SQ8 tensor core distance engine, while TierLift performs source-aware bounded reduction with hybrid DRAM/SSD paths to scale graph consolidation under limited host memory. Experiments on four large-scale benchmarks show that SHEAR runs 1.73–10.04× faster in end-to-end construction than baselines while maintaining competitive ANNS quality.

# References


[1] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore L. Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proc. VLDB Endow.* 16, 11 (2023), 3433–3446. doi:10.14778/3611479.3611537

[2] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1, Article 43 (2025), 31 pages. doi:10.1145/3709693

[3] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2055–2063. doi:10.1109/CVPR.2016.226

[4] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2018. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. arXiv:1611.09268 [cs.CL]

[5] Rihan Chen, Bin Liu, Han Zhu, Yaoxuan Wang, Qi Li, Buting Ma, Qingbo Hua, Jun Jiang, Yunlong Xu, Hongbo Deng, and Bo Zheng. 2022. Approximate Nearest Neighbor Search under Neural Similarity Metric for Large-Scale Recommendation. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM '22)*. Association for Computing Machinery, New York, NY, USA, 3013–3022. doi:10.1145/3511808.3557098

[6] Cohere. 2023. Cohere Wikipedia Dataset (Multilingual v3 Embeddings). https://huggingface.co/datasets/Cohere/wikipedia-2023-11-embed-multilingual-v3.

[7] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry (SCG '04)*. Association for Computing Machinery, New York, NY, USA, 253–262. doi:10.1145/997817.997857

[8] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web (WWW '11)*. Association for Computing Machinery, New York, NY, USA, 577–586. doi:10.1145/1963405.1963487

[9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Trans. Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474

[10] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 8 (2022), 4139–4150. doi:10.1109/TPAMI.2021.3067706

[11] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754

[12] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (2024), 27 pages. doi:10.1145/3654970

[13] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik P. A. Lensch. 2023. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Trans. Big Data* 9, 1 (2023), 267–279. doi:10.1109/TBDATA.2022.3161156

[14] Hyein Gu and Min Soo Kim. 2025. ScaDANN: A Scalable Disk-Based Graph Indexing Method for ANN. In *Proceedings of the Workshops of the EDBT/ICDT 2025 Joint Conference*. ceur-ws.org, 7 pages. https://ceur-ws.org/Vol-3946/TGD-5.pdf

[15] Yuntao Gui, Peiqi Yin, Xiao Yan, Chaorui Zhang, Weixi Zhang, and James Cheng. 2026. PilotANN: Memory-Bounded GPU Acceleration for Vector Search. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 1*. ACM, 348–358. doi:10.1145/3770854.3780192

[16] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, 171–186. https://www.usenix.org/conference/osdi25/presentation/guo

[17] Hao Guo and Youyou Lu. 2026. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. *ACM Trans. Storage* 22, 2, Article 12 (2026), 30 pages. doi:10.1145/3793926

[18] Guo Hao and Lu Youyou. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. USENIX Association, 133–147. https://www.usenix.org/conference/fast26/presentation/guo

[19] Chengying Huan, Renjie Yao, Shaonan Ma, Rong Gu, Zhengyi Yang, Lizheng Chen, Zhibin Wang, Mingxing Zhang, Fang Xi, Guihai Chen, and Chen Tian. 2026. CMANNS: GPU-Accelerated Graph Index Construction for ANNS via Compute–Memory Disaggregation. *Proc. ACM Manag. Data* 4, 3, Article 150 (2026), 27 pages. doi:10.1145/3802027

[20] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[21] Haodi Jiang, Hao Guo, Minhui Xie, Jiwu Shu, and Youyou Lu. 2025. High-Throughput, Cost-Effective Billion-Scale Vector Search with a Single GPU. *Proc. ACM Manag. Data* 3, 6, Article 334 (2025), 27 pages. doi:10.1145/3769799

[22] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2019), 535–547. doi:10.1109/TBDATA.2019.2921572

[23] Sukjin Kim, Seongyeon Park, Si Ung Noh, Junguk Hong, Taehee Kwon, Hunseong Lim, and Jinho Lee. 2025. PathWeaver: A High-Throughput Multi-GPU System for Graph-Based Approximate Nearest Neighbor Search. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 1501–1517. https://www.usenix.org/conference/atc25/presentation/kim

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Curran Associates, Inc., 9459–9474.

[25] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform. In *Proceedings of the 19th International Middleware Conference Industry (Middleware '18)*. Association for Computing Machinery, New York, NY, USA, 9–16. doi:10.1145/3284028.3284030

[26] Liang Li, Shufeng Gong, Yanan Yang, Yiduo Wang, and Jie Wu. 2026. I/O Optimizations for Graph-Based Disk-Resident Approximate Nearest Neighbor Search: A Design Space Exploration. *Proc. VLDB Endow.* 19, 7 (2026), 1484–1498. doi:10.14778/3801059.3801064

[27] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Bocheng Yu, Baihua Zheng, and Yunjun Gao. 2025. Scalable Graph Indexing using GPUs for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 6, Article 311 (2025), 27 pages. doi:10.1145/3769825

[28] Chaoqiang Liu, Xiaofei Liao, Long Zheng, Yu Huang, Haifeng Liu, Yi Zhang, Haiheng He, Haoyan Huang, Jingyi Zhou, and Hai Jin. 2024. L-FNNG: Accelerating Large-Scale KNN Graph Construction on CPU-FPGA Heterogeneous Platform. *ACM Trans. Reconfigurable Technol. Syst.* 17, 3, Article 46 (2024), 29 pages. doi:10.1145/3652609

[29] Chaoqiang Liu, Haifeng Liu, Long Zheng, Yu Huang, Xiangyu Ye, Xiaofei Liao, and Hai Jin. 2023. FNNG: A High-Performance FPGA-based Accelerator for K-Nearest Neighbor Graph Construction. In *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '23)*. Association for Computing Machinery, New York, NY, USA, 67–77. doi:10.1145/3543622.3573189

[30] Yaowen Liu, Xuejia Chen, Anxin Tian, Haoyang Li, Qinbin Li, Xin Zhang, Alexander Zhou, Chen Jason Zhang, Qing Li, and Lei Chen. 2026. GPU-Accelerated Algorithms for Graph Vector Search: Taxonomy, Empirical Study, and Research Directions. arXiv:2602.16719 [cs.DB]

[31] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. 2024. JUNO: Optimizing High-Dimensional Approximate Nearest Neighbour Search with Sparsity-Aware Algorithm and Ray-Tracing Core Mapping. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 549–565. doi:10.1145/3620665.3640360

[32] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 15, 2 (2021), 246–258. doi:10.14778/3489496.3489506

[33] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs. *Inf. Syst.* 45 (2014), 61–68. doi:10.1016/j.is.2013.10.006

[34] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473

[35] Magdalen Dobson Manohar, Zheqi Shen, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '24)*. Association for Computing Machinery, New York, NY, USA, 270–285. doi:10.1145/3627535.3638475

[36] Marius Muja and David G. Lowe. 2009. Fast Approximate Nearest Neighbors with Automatic Algorithm Configuration. In *Proceedings of the Fourth International Conference on Computer Vision Theory and Applications*. SciTePress, 331–340. doi:10.5220/0001787803310340

[37] Javier Alvaro Vargas Muñoz, Marcos André Gonçalves, Zanoni Dias, and Ricardo da Silva Torres. 2019. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search. *Pattern Recognit.* 96 (2019), 106970. doi:10.1016/j.patcog.2019.106970

[38] NVIDIA Corporation. 2017. *CUTLASS: Fast Linear Algebra in CUDA C++*. https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/

[39] NVIDIA Corporation. 2026. *cuVS CAGRA: Augmented Core Extraction Index Build*. https://docs.rapids.ai/api/cuvs/stable/c_api/neighbors_cagra_c/

[40] Naoki Ono and Yusuke Matsui. 2023. Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*. Association for Computing Machinery, New York, NY, USA, 1659–1667. doi:10.1145/3581783.3612290

[41] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4236–4247. doi:10.1109/ICDE60146.2024.00323

[42] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615. doi:10.1007/S00778-024-00864-X

[43] Zhen Peng, Minjia Zhang, Kai Li, Ruoming Jin, and Bin Ren. 2023. iQAN: Fast and Accurate Vector Search with Efficient Intra-Query Parallelism on Multi-Core Architectures. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. Association for Computing Machinery, New York, NY, USA, 313–328. doi:10.1145/3572848.3577527

[44] Neha Prakriya, Yuze Chi, Suhail Basalama, Linghao Song, and Jason Cong. 2024. TAPA-CS: Enabling Scalable Accelerator Design on Distributed HBM-FPGAs. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 966–980. doi:10.1145/3620666.3651347

[45] Tobias Rubel, Richard Wen, Laxman Dhulipala, Lars Gottesbüren, Rajesh Jayaram, and Jakub Łącki. 2026. PiPNN: Ultra-Scalable Graph-Based Nearest Neighbor Indexing. To appear at KDD '26. arXiv:2602.21247 [cs.DB]

[46] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. 2022. LAION-5B: An open large-scale dataset for training next generation image-text models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Curran Associates, Inc., 25278–25294.

[47] Joobo Shim, Jaewon Oh, Hongchan Roh, Jaeyoung Do, and Sang-Won Lee. 2025. Turbocharging Vector Databases using Modern SSDs. *Proc. VLDB Endow.* 18, 11 (2025), 4710–4722. doi:10.14778/3749646.3749724

[48] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. Curran Associates, Inc., 13748–13758.

[49] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuecang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards High-Throughput and Low-Latency Billion-Scale Vector Search via CPU/GPU Collaborative Filtering and Re-Ranking. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, 171–185. https://www.usenix.org/conference/fast25/presentation/tian-bing

[50] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. 2025. BANG: Billion-Scale Approximate Nearest Neighbour Search Using a Single GPU. *IEEE Transactions on Big Data* 11, 6 (2025), 3142–3157. doi:10.1109/TBDATA.2025.3581085

[51] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 188 (2025), 27 pages. doi:10.1145/3725325

[52] Bohai Wang, Yanhao Wang, Huiqi Hu, Weichen Zhao, and Minghao Zhao. 2026. Distance Comparison Operation Optimization in ANNS: A Survey and Experimental Evaluation.. In *Proceedings of the 29th International Conference on Extending Database Technology (EDBT)*. 578–591.

[53] Hui Wang, Wan-Lei Zhao, Xiangxiang Zeng, and Jianye Yang. 2021. Fast k-NN Graph Construction by GPU based NN-Descent. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 1929–1938. doi:10.1145/3459637.3482344

[54] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, New York,

NY, USA, 2614–2627. doi:10.1145/3448016.3457550

[55] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (2025), 29 pages. doi:10.1145/3725260

[56] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1, Article 14 (2024), 27 pages. doi:10.1145/3639269

[57] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978. doi:10.14778/3476249.3476255

[58] Yang Xiao, Mo Sun, Ziyu Song, Bing Tian, Jie Sun, Jie Zhang, Zeke Wang, Zonghui Wang, Wenzhi Chen, and Fei Wu. 2026. FlashANNS: GPU-Driven Asynchronous I/O Pipelining for Eliminating Storage-Compute Bottlenecks in Billion-Scale Similarity Search. *Proc. ACM Manag. Data* 4, 1, Article 38 (2026), 27 pages. doi:10.1145/3786652

[59] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 6 (2025), 1825–1838. doi:10.14778/3725688.3725709

[60] Xizhe Yin, Chao Gao, Zhijia Zhao, and Rajiv Gupta. 2025. PANNS: Enhancing Graph-based Approximate Nearest Neighbor Search through Recency-aware Construction and Parameterized Search. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '25)*. Association for Computing Machinery, New York, NY, USA, 369–381. doi:10.1145/3710848.3710867

[61] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated Proximity Graph Approximate Nearest Neighbor Search and Construction. In *2022 IEEE 38th International Conference on Data Engineering*. IEEE, 552–564. doi:10.1109/ICDE53745.2022.00046

[62] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, and Yu Wang. 2023. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 283–296. doi:10.1145/3613424.3614292

[63] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Vector Query Processing for Large Datasets Beyond GPU Memory with Reordered Pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 23–40. https://www.usenix.org/conference/nsdi24/presentation/zhang-zili-pipelining

[64] Haoru Zhao, Jingkai He, Jingyao Zeng, Mingkai Dong, and Dong Du. 2026. Disentangling Graph Dependencies for Efficient Billion-Scale GPU Vector Search. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1563–1584. https://www.usenix.org/conference/osdi26/presentation/zhao

[65] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1033–1044. doi:10.1109/ICDE48307.2020.00094

[66] Yuhao Zhu. 2022. RTNN: accelerating neighbor search using hardware ray tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 76–89. doi:10.1145/3503221.3508409